tham

树状数组

热身题1

有一个n个数的序列$\{a_n\}$，要求实现三种操作：

1. 将a_x改为y；
2. 将a_x加上y；
3. 查询$\sum_{i=l}^{r} a_i$的值。

$1 \le n, q \le 10^5$

热身题2

有一个n个数的序列$\{a_n\}$，要求实现两种操作：

1. 将$a_x(l \le x \le r)$加上y；
2. 查询$\sum_{i=l}^{r} a_i$的值。

$1 \le n, q \le 10^5$

热身题3

有一个n个数的序列$\{a_n\}$，要求实现两种操作：

1. 令$a_i = c$（$l \le i \le r$）
2. 查询$\sum_{i=l}^{r} a_i$的值。

$1 \le n, q \le 10^5$

热身题4 / 5

有一个n个数的序列$\{a_n\}$，要求实现三种操作：

1. 将$a_x(l \le x \le r)$改为y；
2. 将$a_x(l \le x \le r)$加上y；
3. 查询$\sum_{i=l}^{r} a_i$的值。

$1 \le n, q \le 10^5$

有一个n个数的序列$\{a_n\}$，要求实现四种操作：

1. 将$a_x(l \le x \le r)$改为y；
2. 将$a_x(l \le x \le r)$加上y；
3. 将$a_x(l \le x \le r)$乘上y；
4. 查询$\sum_{i=l}^{r} a_i$的值。

$1 \le n, q \le 10^5$

引言

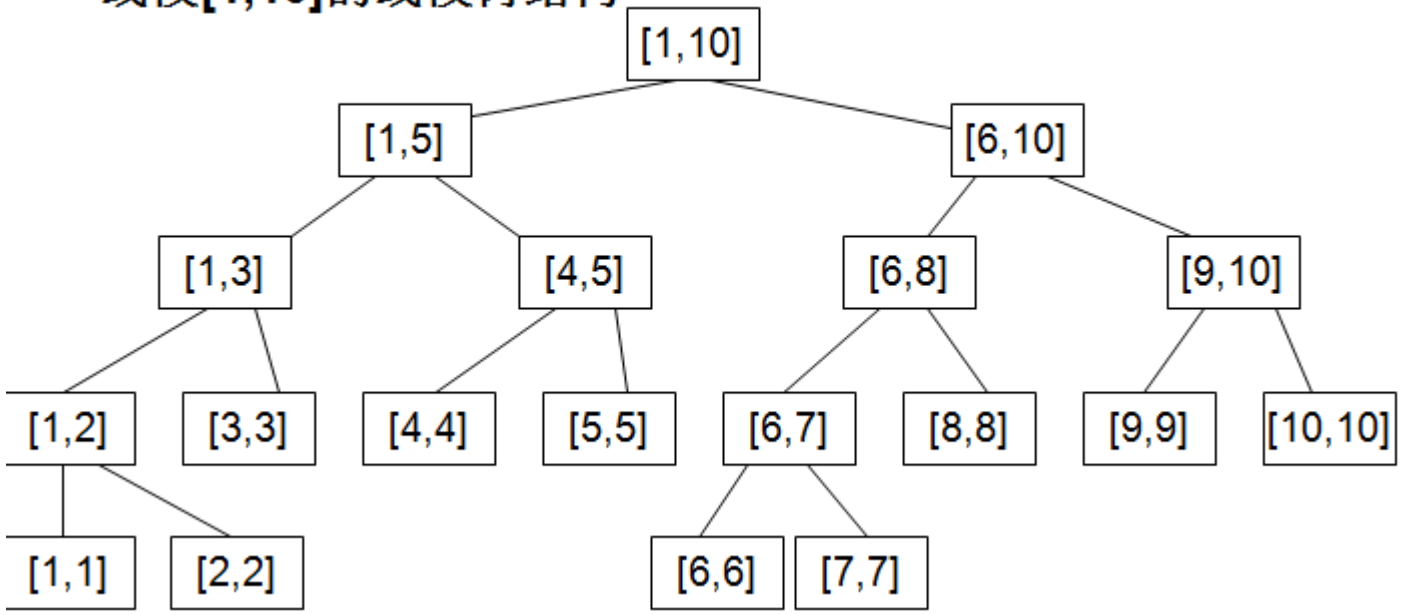
- 在竞赛中，经常会有一些跟区间有关的问题，例如，给一些区间线段求并区间的长度，或者并区间的个数等等。这些问题的描述都非常简单，但是通常情况下数据范围会非常大，用朴素方法的时间复杂度过高，导致不能在规定时间内得到问题的解。这时，就需要一种高效的数据结构来处理这样的问题。

- 线段树是一种基于分治思想的数据结构
- 分治，即分而治之，将问题实例划分成子实例，并分别递归地解决每个子实例，最后将子实例的解组合起来。比如，二分查找。
- 线段树是一种二叉树结构，操作都是递归实现的
- 特点：将线段组织成树，在树中对线段进行处理

线段树（SEGMENT TREE）简介

- **线段树是一种怎样的树形结构?**
- 线段树是一种二叉搜索树，每个结点度小于等于二，即每个结点最多有两颗子树，线段树的每个结点都存储了一个区间，也可以理解成一个线段。与区间树相似，它**将一个区间划分成一些单元区间，每个单元区间对应线段树中的一个叶结点**。使用线段树可以快速的查找某一个节点在若干条线段中出现的次数，时间复杂度为O(logN)。
- 线段树的空间复杂度一般为O(2N)，实际应用时一般还要开4N的数组以免越界，因此有时需要离散化让空间压缩。

线段[1,10]的线段树结构



线段树的结构与性质

- 每个节点都是一个[a,b]的区间，根节点代表了整个所要处理的区间
- 对于每个非叶节点[a,b]，令mid = (a + b) / 2；则其左右儿子节点代表的区间为[a,mid]，[mid+1,b]

- 二叉树结构：线段树是一个平衡树，树的高度为logN
- 线段树把区间上的任意一条长度为L的线段都分成不超过2logL条线段的并集

- 任两个结点要么是包含关系要么没有公共部分，不可能部分重叠，每层节点的区间并即为全体区间

- 给定一个叶子p，从根到p路径上所有结点代表的区间都包含点p，且其他结点代表的区间都不包含点p

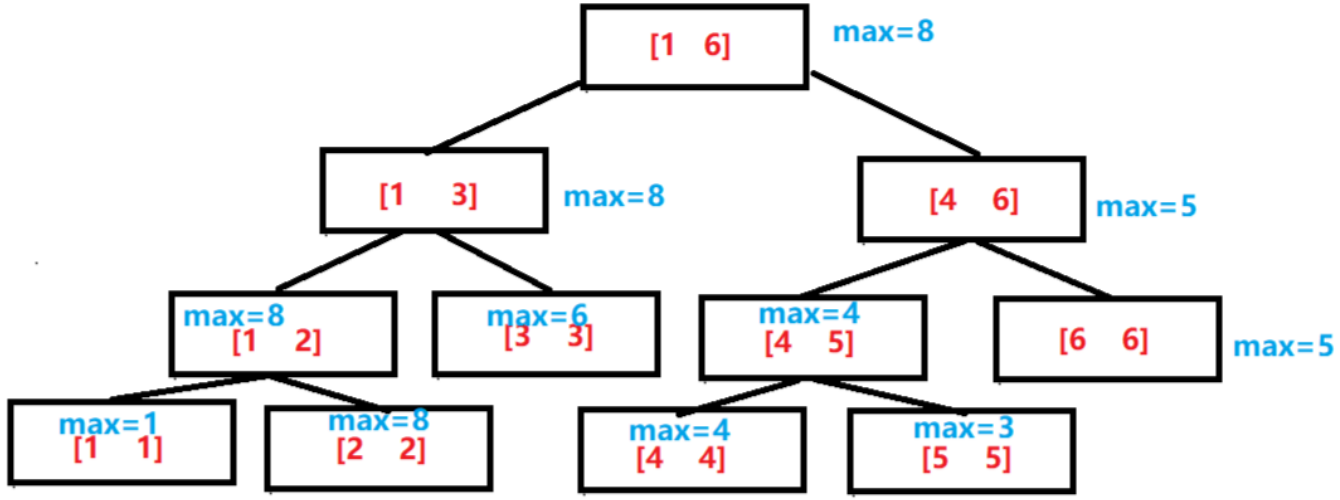
线段树的一些性质

- 线段树的适用范围很广，可以在线维护修改以及查询区间上的最值，求和。更可以扩充到二维线段树（矩阵树）和三维线段树（空间树）。对于一维线段树来说，每次更新以及查询的时间复杂度为O(logN)。
- **线段树和其他RMQ算法的区别？**
- 常用的解决RMQ问题有ST算法，二者预处理时间都是O(NlogN)，而且ST算法的单次查询操作是O(1)，看起来比线段树好多了，但二者的区别在于线段树支持在线更新值（修改），而ST算法不支持在线操作（修改）。
- 这里也存在一个误区，刚学线段树的时候就以为线段树和树状数组差不多，用来处理RMQ问题和求和问题，但其实线段树的功能远远不止这些，要熟练的理解线段这个概念才能更加深层次的理解线段树。

- https://www.cnblogs.com/xenny/p/9801703.html
- https://blog.csdn.net/qq_35811331/article/details/96982758

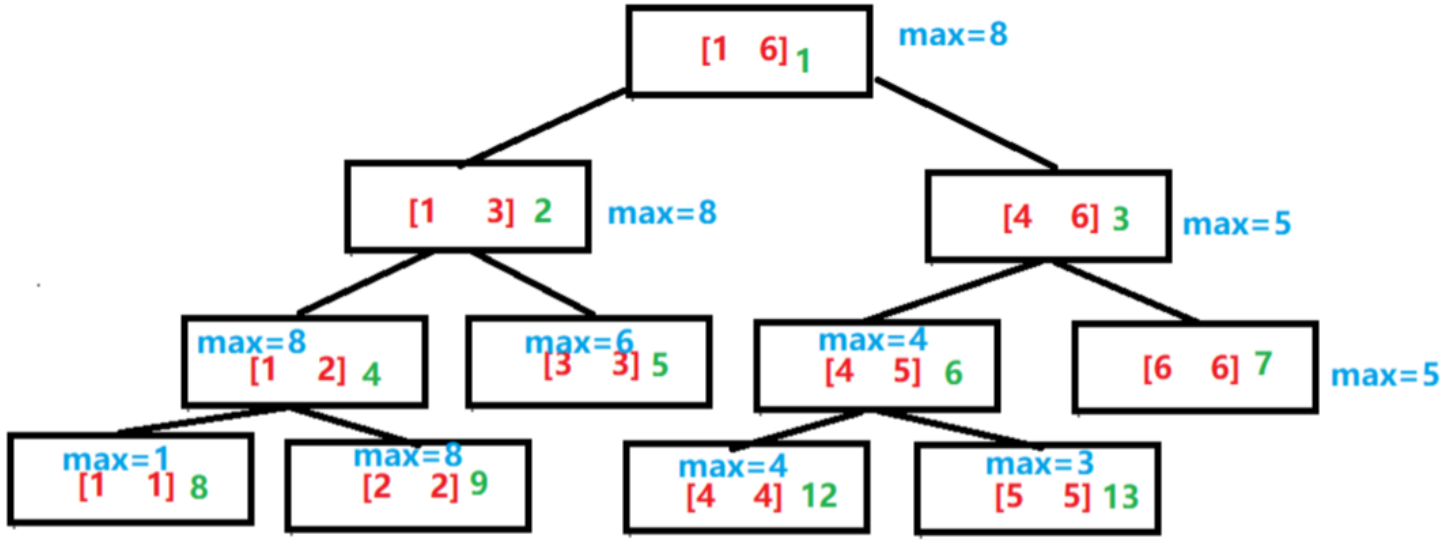
线段树的基本内容

- 对于一棵线段树首先需要知道：每个结点存什么，结点下标是什么，如何建树？
- 对于A[1:6] = {1,8,6,4,3,5}，以区间最大值来阐述上面的三个概念，线段树结构如图所示，红色代表每个结点存储的区间，蓝色代表该区间最值。



- 可以发现，**每个叶子结点的值就是数组的值，每个非叶子结点的度都为二，且左右两个孩子分别存储父亲一半的区间。每个父亲的存储的值也就是两个孩子存储的值的最大值。**

- 线段树的结点到底是如何存储区间的呢？以及如何快速找到非叶子结点的孩子以及非根节点的父亲呢？如同树状数组你理解了lowbit就能很快理解树状数组一样，线段树你只要理解了结点与结点之间的关系便能很快理解线段树的基本知识。
- 线段树的一个节点存储一个区间的信息，对于一个区间[l,r]来说，最重要的数据当然就是区间的左右端点l和r，以及区间内要维护的内容。
- 对于前面的线段树，增加绿色数字为线段树每个结点的下标

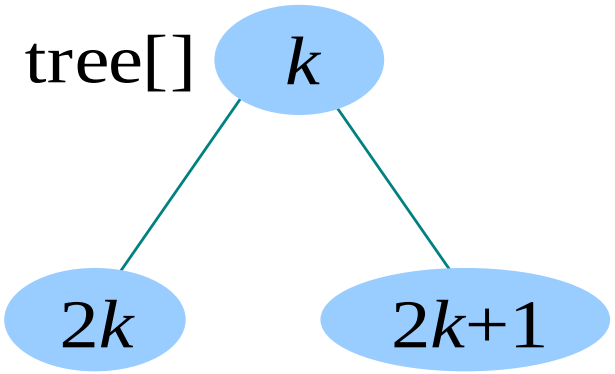


- 线段树可看成一个完全二叉树。

线段树的存储方式

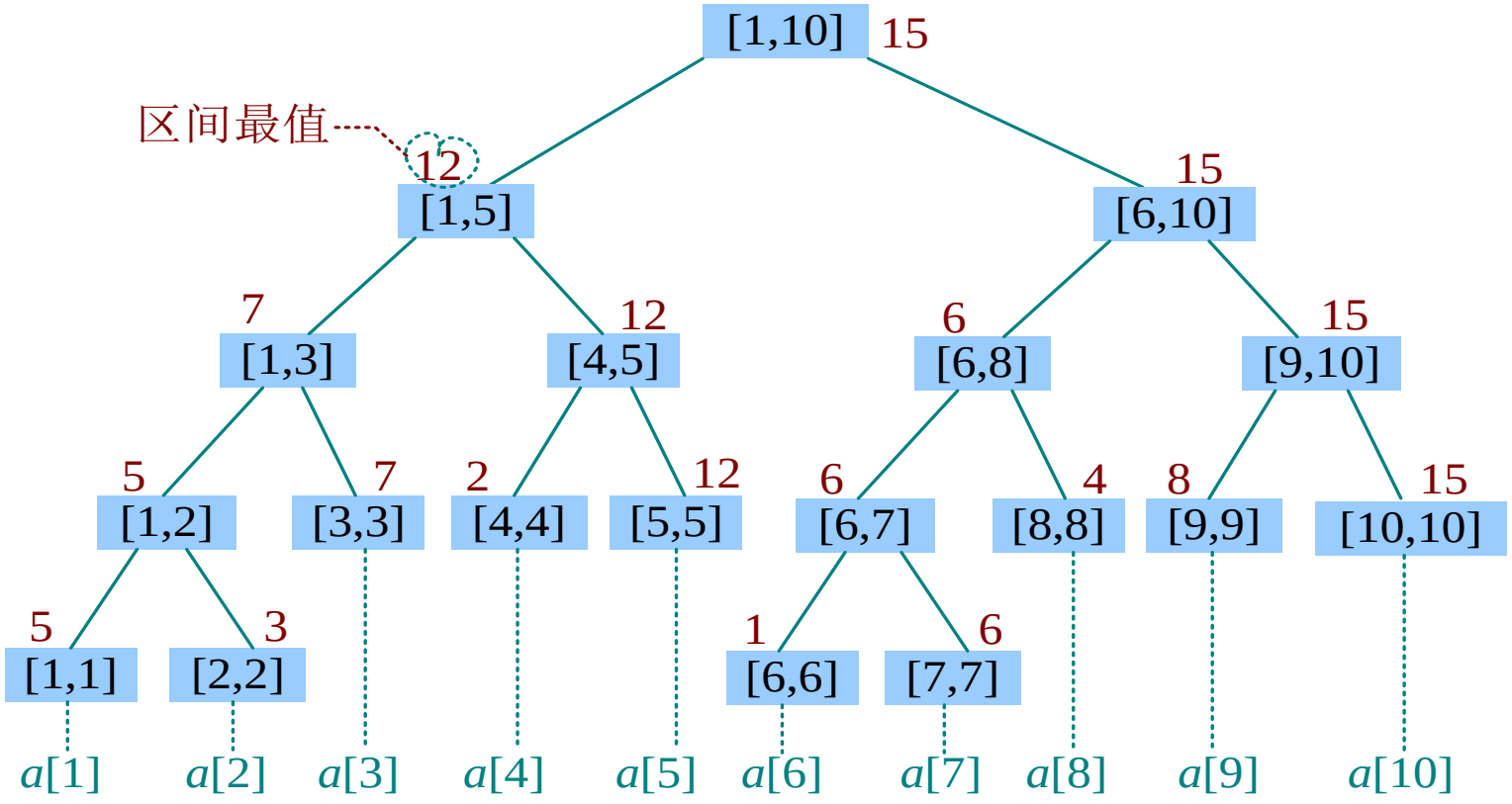
对于区间最值（最大值或最小值）查询问题，线段树的每个节点包含三个域：l、r、mx，其中l和r表示区间的左右端点，mx表示$[l, r]$区间最值。

线段树除了最后一层，其他层构成一棵满二叉树，因此采用顺序存储方式，用一个数组tree[]存储节点。

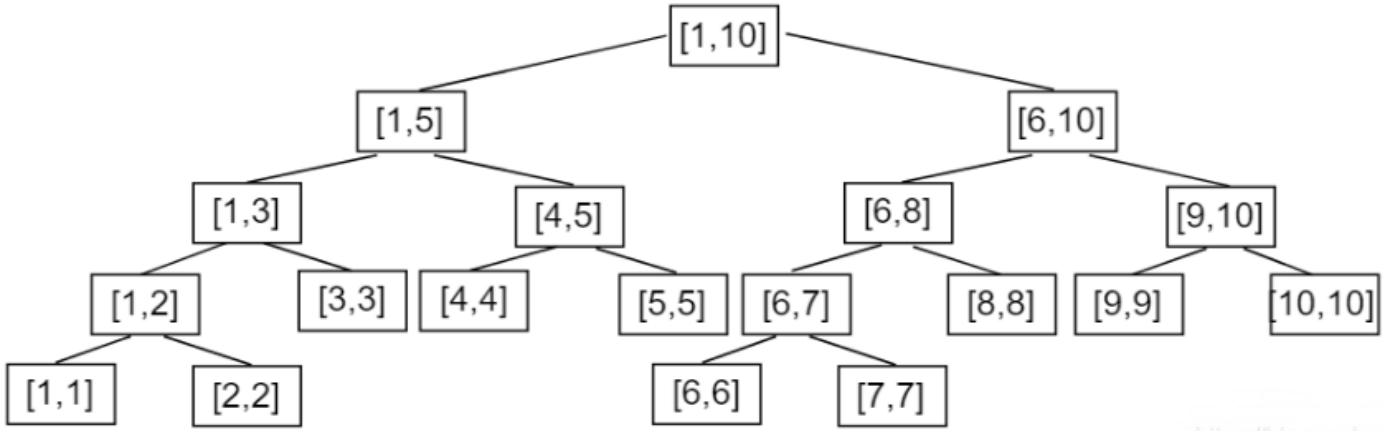


线段树

以最大值为例，10个元素$a[1..10]=\{5,3,7,2,12,1,6,4,8,15\}$，线段树如下。



线段树的空间为什么要开4N



- 1、如果区间长度为N，那么构造的线段树的叶子结点个数就是N
- 2、深度为k-1的满二叉树有2^{k-1} -1个结点，且第k层的节点数是: 2^{k-1}
- 3、假设线段树最后一层的叶子结点为m，倒数第二层叶子结点为n，（n+m=N）由1、2可知，线段树结点个数为：根据 m + n = N，则倒数第二层的结点个数 m/2+n，很显然第一层到倒数第二层的结点个数总和为 2*(m / 2 + n) – 1，
  二叉树的总结点数：2*(m / 2 + n) - 1 + m = m + 2 * n - 1 + m = 2 * (m + n) - 1 = 2*N - 1 （但是空间要开4N ？）
- 很明显上面的计算方法，漏掉了最后的一层中间会存在一些空的元素，虽然没有使用，但是为保证完全二叉树的特性，也要留出空间。因此，线段树的最后一层全部补满则最后一层有2*(m/2+n)个节点，总节点数4*(m/2+n)-1个节点，因此最坏的情况下开4N空间足够。
- https://blog.csdn.net/qq_43803508/article/details/97501894

线段树的存储

- 线段树是一个树形结构，其信息都是保存在树的节点中，可以使用结构体的方式建立节点，一个节点的基本结构如下：

```
struct node{
    int left ,right, mid;
};
```

- 这只是线段树节点的最基本结构，在具体解题中，还需要在节点中添加其他数据域保存信息，并且线段树中每个节点对应原序列的一段区间，递推地进行区间操作，有时候为了代码方便节点的区间边界left、right可单独拿出来作为递归函数的参数进行传递。

创建线段树

可以采用递归的方法创建线段树。

算法步骤：

（1）若是叶子节点（$l=r$），则节点的最值就是对应位置的元素值。

（2）若是非叶子节点，则递归创建左子树和右子树。

（3）节点的区间最值等于该节点左右子树最值的最大值。

线段树

算法实现

线段树存储有两种方法：

- 携带区间信息
- 不携带区间信息

```
const int maxn = 100005;
int a[maxn],t[maxn<<2];          //a[]为原序列数组，t[]为线段树

void Pushup(int k){          //更新函数，这里是实现最大值 ，同理可以变成，最小值，区间和等
    t[k] = max(t[k<<1],t[k<<1|1]);
}

//递归方式建树 build(1,1,n);
void build(int k,int l,int r){     // k为线段树当前需要建立的结点，
                                   // l为当前需要建立区间的左端点，r则为右端点
    if(l == r)     //左端点等于右端点，即为叶子节点，直接赋值即可
        t[k] = a[l];
    else{
        int m = l + ((r-l)>>1);     //m则为中间点，左儿子的结点区间为[l,m],右儿子的结点区间为[m+1,r]
        build(k<<1,l,m);            //递归构造左儿子结点
        build(k<<1|1,m+1,r);       //递归构造右儿子结点
        Pushup(k);            //递归回溯时，从儿子结点更新父结点信息
    }
}
```

单点更新

单点更新指修改一个元素的值，例如将$a[i]$修改为v。

算法步骤：

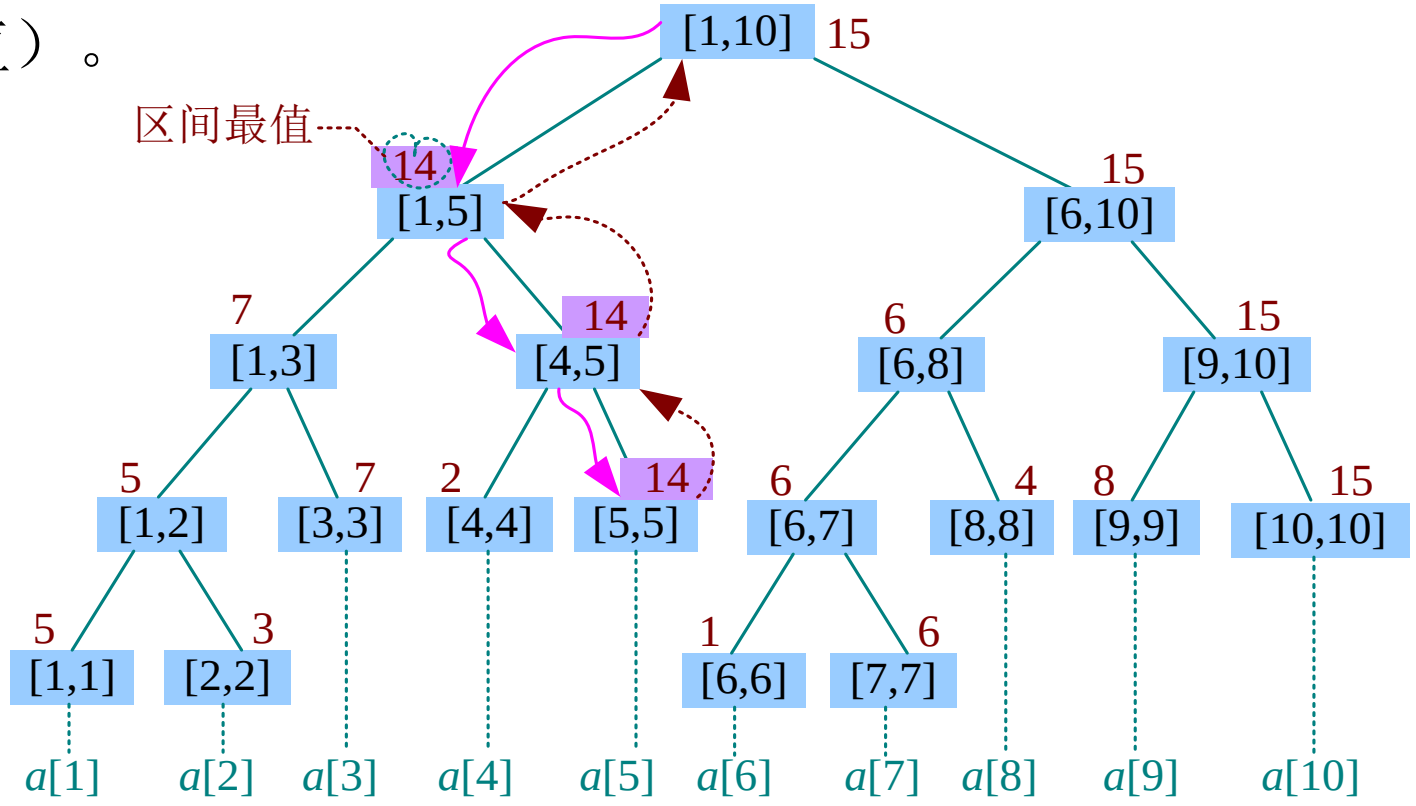
（1）若是叶子节点，满足$l=r$且$l=i$，则修改该节点的最值为v。

（2）若是非叶子节点，则判断是在左子树中更新还是在右子树中更新。

（3）返回时更新节点的最值。

线段树

例如，修改第5个节点的值为14，先从树根向下找第5个元素所在的叶子节点，将其最值修改为14，**返回时更新路径上所有节点的最值**（左右子节点最值的最大值）。



算法实现

```
//递归方式更新 updata(p,v,1,n,1);
void updata(int p,int v,int l,int r,int k){     // p为原序列数组a[]下标，v为要加上的值，
                                                // l，r为线段树结点区间，k为线段树结点下标
    if(l == r)      // 左端点等于右端点，即为叶子结点，直接加上v即可
        a[p] += v,t[k] += v;     // 原数组a[]中的值 和 线段树数组t[]中的值都进行更新
    else{
        int m = l + ((r-l)>>1);     //m则为中间点，左儿子的结点区间为[l,m],右儿子的结点区间为[m+1,r]
        if(p <= m)                  //如果需要更新的结点在左子树区间
            updata(p,v,l,m,k<<1);
        else                        //如果需要更新的结点在右子树区间
            updata(p,v,m+1,r,k<<1|1);
        Pushup(k);                  //更新父节点的值
    }
}
```

区间查询

区间查询指查询一个$[l, r]$区间的最值。

算法步骤：

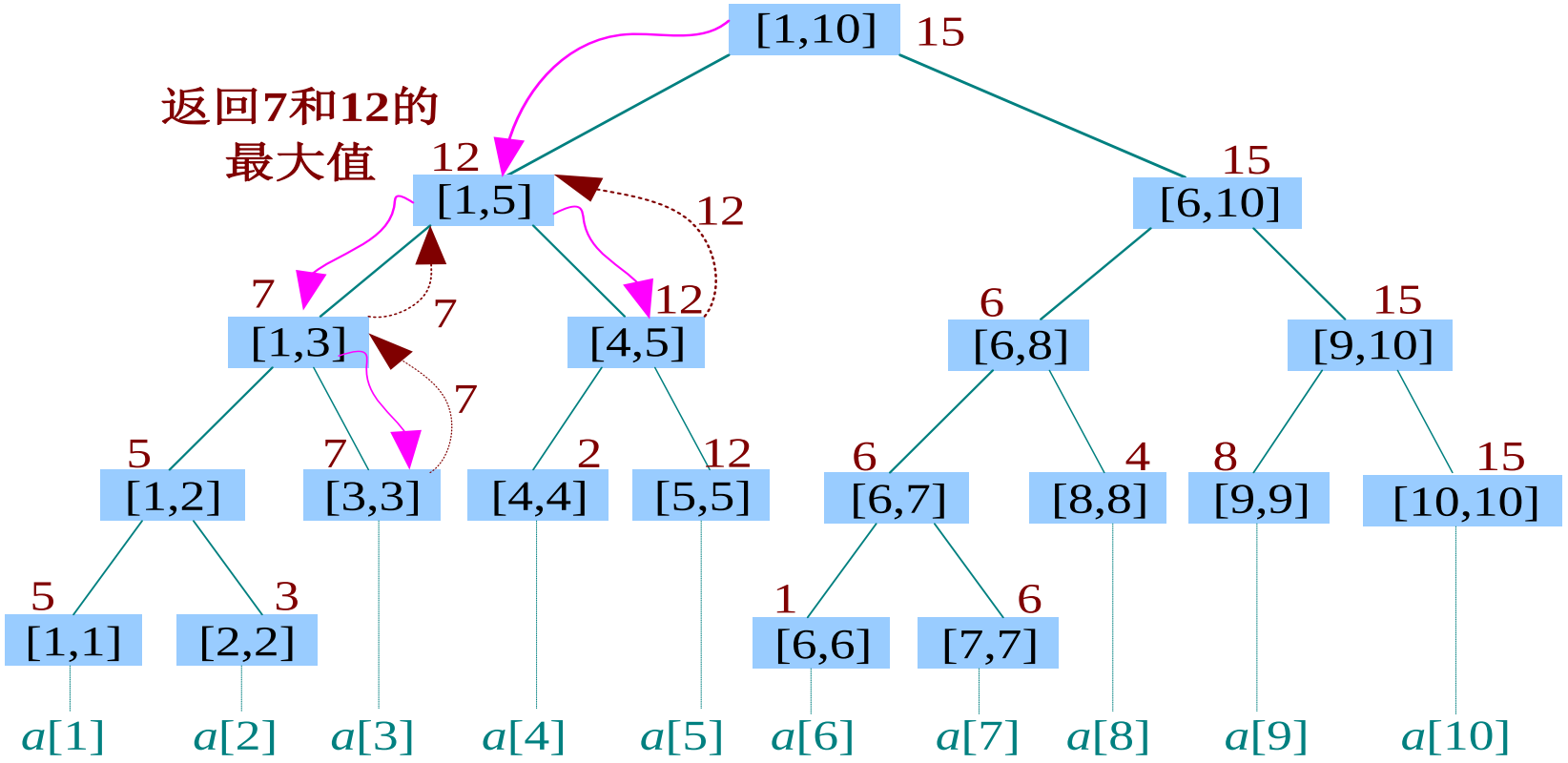
（1）若节点所在的区间被查询区间$[l, r]$覆盖，则返回该节点的最值。

（2）判断是在左子树中查询，还是在右子树中查询。

（3）返回最值。

线段树

例如，在[1,10]的线段树中查询[3,5]区间的最值，过程如下。



线段树

算法实现

区间查询有两种方法：

- 区间覆盖
- 区间相等

```
//递归方式区间查询 query(L,R,1,n,1);
int query(int L,int R,int l,int r,int k){    // [L,R]即为要查询的区间，l，r为线段树结点区间，k为线段树结点下标
    if(L <= l && r <= R)      // 如果当前结点的区间真包含于要查询的区间内，则返回结点信息且不需要往下递归
        return t[k];
    else{
        int res = -INF;     //返回值变量，根据具体线段树查询的什么而自定义
        int mid = l + ((r-l)>>1);    // m则为中间点，左儿子的结点区间为[l,m],右儿子的结点区间为[m+1,r]
        if(L <= m)          // 如果左子树和需要查询的区间交集非空
            res = max(res, query(L,R,l,m,k<<1));
        if(R > m)           // 如果右子树和需要查询的区间交集非空，注意这里不是 else if，
                            // 因为查询区间可能同时和左右区间都有交集
            res = max(res, query(L,R,m+1,r,k<<1|1));

        return res;    //返回当前结点得到的信息
    }
}
```

算法分析：

树上的操作大多与树高相关。因为线段树对区间进行二分，是一棵平衡二叉树，树高为$O(\log n)$，查询和更新的时间复杂度均为$O(\log n)$，空间复杂度为$O(n)$。线段树主要用于更新和查询，一般至少有一个是区间更新或查询。更新和查询的种类变化多样，灵活运用线段树可以解决多种问题。

线段树中的懒标记：

- 区间更新
- 区间查询

区间更新

区间更新指修改一整个区间元素的值，例如将 $a[i]$~a[j] 增加 v。

算法步骤：

（1）若是叶子节点，满足$l=r$且$l=i\sim j$，则修改该节点的值+v。

（2）若是非叶子节点，则在左子树和右子树中递归更新。

（3）返回时，回溯更新节点的信息。

线段树的区间更新

- 树状数组中的区间更新利用了差分的思想，而线段树的区间更新相对于树状数组就稍微复杂一点，这里引入一个新东西，Lazy_tag，字面意思就是懒惰标记的意思，实际上它的功能也就是偷懒，**因为对于一个区间[L,R]来说，如果每次都要更新到区间中的每个值（线段树中的叶子结点），则这样单次更新的复杂度将会是O(NlogN)**，这太高了，所以引进了Lazy_tag，这个标记一般用于处理线段树的区间更新。

  线段树在进行区间更新的时候，为了提高更新的效率，所以**每次更新只更新到“更新区间”完全覆盖线段树结点区间为止**，这样使得被更新结点的子孙结点的区间可以延迟更新，即需要在线段树的被更新结点上打上一个标记，称为lazy-tag，等到下次访问这个结点的子结点时再将这个标记传递给子结点，所以也可以叫延迟标记。
- 也就是说递归更新的过程，更新到结点区间为需要更新的区间的真子集不再往下更新，下次若是遇到需要用这下面的结点的信息，再去更新这些结点，所以这样的话使得区间更新的操作和区间查询类似，复杂度为O(logN)。

```
void Pushdown(int k){     // 更新子树的 lazy值
    if(lazy[k])           // 如果线段树的结点上有 lazy标记
    {
        lazy[k<<1] += lazy[k];      //更新左子树的lazy值
        lazy[k<<1|1] += lazy[k];    //更新右子树的lazy值
        t[k<<1] += lazy[k];         //左子树的最值加上lazy值
        t[k<<1|1] += lazy[k];       //右子树的最值加上lazy值
        lazy[k] = 0;                //Lazy标记下放后，  lazy值归0
    }
}

//递归更新区间 updata(L,R,v,1,n,1);
void updata(int L,int R,int v,int l,int r,int k)
{                                   //[L,R]即为要更新的区间，l，r为线段树结点区间，k为线段树结点下标
    if(L <= l && r <= R){           //如果当前结点的区间真包含于要更新的区间内
        lazy[k] += v;               //打上懒标记
        t[k] += v;                  //最大值加上v之后，此区间的最大值也肯定是加v
    }
    else{
          Pushdown(k);              //重难点，查询线段树这个结点的 lazy标记，更新子树
          int m = l + ((r-l)>>1);
          if(L <= m)                //如果左子树和需要更新的区间交集非空
              update(L,R,v,l,m,k<<1);
          if(m < R)                 //如果右子树和需要更新的区间交集非空
              update(L,R,v,m+1,r,k<<1|1);
          Pushup(k);                //更新父节点
    }
}
```

- 注意Pushdown这个函数，也就是当需要查询某个结点的子树时，需要用到这个函数，函数功能就是更新子树的lazy值，可以理解为平时先把事情放着，等到哪天要检查的时候，就临时再去做，而且做也不是一次性做完，检查哪一部分它就只做这一部分。
- 值得注意的是，使用了Lazy_tag后，区间查询也需要改变。区间查询的代码则变为：

```
//递归方式区间查询 query(L,R,1,n,1);
int query(int L,int R,int l,int r,int k) // [L,R]即为要查询的区间，l，r为线段树结点的区间，k为线段树结点下标
{
    if(L <= l && r <= R)          //如果当前结点的区间真包含于要查询的区间内，则返回结点信息且不需要往下递归
        return t[k];
    else{
        Pushdown(k);        /**每次查询前，都需要先下放结点的 lazy，更新子树的Lazy标记*/
        int res = -INF;     //返回值变量，根据具体线段树查询的什么而自定义
        int mid = l + ((r-l)>>1);     //m则为中间点，左儿子的结点区间为[l,m],右儿子的结点区间为[m+1,r]
        if(L <= m)                    //如果左子树和需要查询的区间交集非空
            res = max(res, query(L,R,l,m,k<<1));
        if(R > m)                     //如果右子树和需要查询的区间交集非空，注意这里不是else if，
                                      //因为查询区间可能同时和左右区间都有交集
            res = max(res, query(L,R,m+1,r,k<<1|1));

        return res;     //返回当前结点得到的信息
    }
}
```

线段树的其他操作

- 通过上述线段树处理区间最值的所有操作，那么转变成求最小值以及区间和问题应该也能很快解决，请手动再实现一下查询区间最小值的线段树和查询区间和的线段树。
- 上述我们只是再讲线段树处理裸区间问题，但是大部分问题不会是让你直接更新查询，而是否真正理解线段树便在于思维是否能从区间跳到线段。

- 区间只是一个线段的一小部分，还有一些非区间问题也可以演变成一段一段的线段，然后再通过线段树进行各种操作。类似二维树状数组，线段树也有二维的结构。

练习

- 洛谷P2345 奶牛集会
- 洛谷P3660 Why Did the Cow Cross the Road III G
- 洛谷P3608 Balanced Photo平衡的照片
- 洛谷P3253 删除物品
- CF1234D Distinct Characters Queries
- POJ2299-Ultra-QuickSort
- P1168 中位数　尝试树状数组解决
- CF1354D Multiset
- P4054 [JSOI2009]计数问题
- P4514 上帝造题的七分钟